Confidence: High.

15. Log Probability.

Đã implement sentence_log_probability() ở implement.py

- log probability giúp tránh vấn đề numerical underflow:
  - + Xác suất điều kiện $P(w_t|context)$ luôn nằm trong khoảng [0,
  khi câu dài, việc nhân liên tiếp các số thực nhỏ hơn 1 làm tích P(S) tiến rất nhanh về 0.
  - + Hàm log là hàm đồng biến, khi biến đổi phép nhân chuỗi số thực thành phép cộng các số âm, máy tính sẽ lưu trữ số dưới dạng tổng số âm cực kỳ ổn định.

16. Experiment 2 – MLE vs Smoothing.

Tại sao kết quả lại như vậy?

- MLE có Valid PPL và Test PPL bằng infinity do hiện tượng Zero Probability. Với MLE, $P(w|h) = 0$

$\Rightarrow \log P = -\infty \Rightarrow$ Perplexity = ~~∞~~ Infinity

- Laplace Smoothing giúp giải quyết được vấn đề trên tập Val/Test vì Smoothing gán 1 lượng xs nhỏ (+1) cho tất cả các n-gram trong vocab

Do đó, mọi unseen n-gram đều có xs > 0, giúp Perplexity có giá trị hữu hạn và mô hình đánh giá được trên dl thực tế.

- Train PPL của Laplace Smoothing cao hơn Train PPL của MLE do Laplace Smoothing trích bớt xs từ các n-gram đã xuất hiện ñ trong Train để cho các unseen ~~ng~~ n-gram.

- Trigram Laplace có PPL lớn hơn Bigram Laplace trên tập Val/Test vì Laplace Smoothing giả định mọi n-gram đều xuất hiện 1 lần. Nhưng ° gian Trigram (V^3) lớn hơn rất nhiều so với Bigram (V^2), làm cho việc +1 phân bổ quá nhiều xác suất cho vô số Trigram không hợp lệ.

19. Experiment 3: Perplexity:

- Training perplexity thay đổi do khi chuyển từ Unigram → Bigram, Training PPL giảm mạnh do thông tin ngữ cảnh từ liền trước. Tuy nhiên, với Laplace Smoothing trên tập Train nhỏ, Trigram PPL lại tăng cao hơn Bigram, do Smoot chiếm quá n̄ xs để phân bổ cho V^3 tổ hợp Trigram.

- Val/Test Perplexity có thể không cải thiện do dl Val/Test luôn chứa nhiều cụm từ mới chưa từng thấy trong tập Train, với Trigram, tỉ lệ n-gram unseen quá lớn làm mô hình bị rơi nặng khiến Train/Test PPL tăng vọ

- Dấu hiệu Overfitting thể hiện ~~qua~~ khi khoảng cách giữ Train PPL và Val/Test PPL ngày càng rộng khi tăng n. Mô hình bậc cao (Trigram) ghi nhớ quá mức cấu tr dl của Train set nhx mất khả năng tổng quát hoá trên Val/Test set

- Tác động của Corpus size:
  + Corpus nhỏ: Bigram cho PPL tốt hơn Trigram trên tập Val/Test
  + Corpus rất lớn: Trigram vượt trội hơn Bigram trên cả tập Val và tập ~~Set~~ Test

23. Một thí nghiệm rất quan trọng.

Context dài hơn không luôn tốt hơn vì:

+ Training Perplexity: khi n tăng (~~Bigr~~ Unigram → Bigram → Trigram), training PPL giảm dần, Trigram có Training PPL thấp nhất vì mô hình dễ dàng "học thuộc lòng" các chuỗi từ xh trong tập Train

+ Val/Test Perplexity: từ Unigram lên Bigram, PPL trên tập Val/Test giảm đáng kể vì Bigram tận dụng được $+^2$ ngữ cảnh từ liền trước.

24. Reflection.

Câu 1:

Khi n tăng, mô hình nhận thêm ngữ cảnh lịch sử dài hơn (thêm 1 từ đứng liền trước), điều này giúp cải thiện mô hình.

Câu 2:

Khi n tăng, không gian trạng thái bùng nổ quá nhanh khiến corpus không thể đại diện hay bao phủ hết các tổ hợp này, dẫn đến phần lớn các N-gram thực tế sẽ có tần suất xuất hiện = 0

Câu 3:

Smoothing phân bổ lại phần xác suất nhỏ cho các N-gram chưa thấy để mô hình đánh giá dữ liệu mới 1 cách ổn định.

Câu 4:

PPL đo độ "bối rối" của mô hình khi ~~đo~~ dự đoán từ tiếp theo trong chuỗi. PPL càng thấp thì mô hình gán xác suất càng cao cho văn bản đánh giá.

Câu 5:

Không, một model có PPL thấp hơn không ~~có~~ luôn tạo ra văn bản tốt hơn đối với con người do PPL chỉ đo khả năng khớp phân phối xs trên tập dl đánh giá, không có tính tự nhiên hay ngữ nghĩa toàn cục.

Câu 6:

Mô hình N-gram không hiểu ngữ nghĩa do nó xem các từ là các ký hiệu rời rạc độc lập, không nhận biết được sự tương đồng giữa các từ gần nghĩa và không có khả năng tổng quát hoá

Câu 7:

Không. Vì mô hình Trigram chỉ sử dụng giả định Markov bậc 2, nên Trigram chỉ sử dụng duy nhất 2 từ liền trước để dự đoán từ tiếp theo và hoàn toàn bỏ qua thông tin của 97 từ đầu tiên.